Duplication, Near-Duplication, and Constraint Integration in Hierarchical Multi-Agent Systems

Anonymous

Anonymous submission draft

Abstract. Similar agent reports can restate one observation, describe fresh executions, or contain complementary requirements whose differences determine a decision. We ask when repetition protects information from summary loss and when it obscures meaningful differences. We develop a controlled benchmark for these distinctions in hierarchical language-model systems. The primary task is to select the feasible candidate schedule with minimum makespan from a supplied set. Executable verifiers determine constraint satisfaction and the correct decision, and an independently implemented checker validates small integer instances. Paired interventions change numerical limits, scoped identities, or dependencies; a separate rule-workflow population tests quantifiers, conditions, and explicit revisions. Frozen reports support comparisons between single presentations, exact copies, and claim-preserving paraphrases under flat and summary-only hierarchical aggregation. The protocol compares ordinary synthesis, required-task checklists, and scope-aware consolidation with common information access and output contracts. This working manuscript currently reports development and interface validation rather than completed held-out hypothesis tests. Initial execution exposed formatting failures and insufficient baseline task competence. After interface validation, the completed one-model pilot showed ceiling flat scheduling accuracy and beneficial repetition effects in hierarchical scheduling, with mixed secondary-workflow findings. An exploratory trace audit separates numerical objective loss from errors after correct constraint delivery. A dated amendment tests repetition targets and summary access. We document these limitations, preserve unsuccessful outputs, and specify paired cluster-level analyses and additional controls needed to distinguish repetition effects from information loss, missing coverage, and resource changes. Human scientific review and the remaining experimental stages are required before submission.

Keywords: multi-agent systems, information integration, duplication, scheduling, behavioral evaluation

1 Status and Research Question

Working research draft; not ready for submission. The implemented benchmark and completed development experiments are distinct from the uncompleted held-out replay. No held-out result is claimed. The following study stages do not yet have complete analysis outputs: main-v7, supporting-v7, live-v8. Human scientific review, interpretation and submission eligibility remain pending. The human author's scientific review is pending. The title is provisional until the findings justify a final contribution.

We ask when repetition protects information from loss in hierarchical agent systems, and when it leads them to overlook contradictions, scope differences, exceptions or updates. The revised working hypothesis permits benefits, harms, negligible effects and mixtures; it is not a conclusion. One possible failure is unnecessary sensitivity to repeated information. Another is failure to respond to similar information whose scope or value changes a mandatory constraint. Neither behavior follows simply from report similarity: two messages can use the same words about different resources, while different words can express the same claim from the same source.

The candidate contribution is a reproducible, controlled test of complementary constraint integration and preservation of scope differences across restricted summary boundaries. We do not propose evidence double counting, provenance, paraphrase sensitivity, or hierarchical communication as new discoveries. Conclusions must remain specific to evaluated models, tasks, interfaces, and resource conditions; deployment prevalence and general intelligence are outside the study's scope.

2 Related Work and Remaining Distinction

Gao et al. [4] are the closest comparison. Their controlled sensing, web-document, and agent-written-log experiments measure how restatements affect beliefs and decisions. Their hierarchy and summary conditions already address communication structure. In their agent-team testbed, the aggregator receives the full log. Our proposed distinction is a conjunction of scoped requirements passed through managers whose root receives summaries without the raw reports. A generic finding that repeated evidence shifts an answer would not establish this distinction.

Bara [1] separates ancestry from report similarity and models correlated extraction errors. We therefore distinguish additional verification executions from new evidence sources and do not infer independence from fresh model calls. HiddenBench [6] examines information distributed among agents and failures to elicit unshared facts. Our frozen replay supplies the complete required report set before aggregation, allowing delivery failures to be separated from later summary loss. MAST [2] provides a broad taxonomy of multi-agent failure; our deterministic, paired interventions address a narrower mechanism.

The current scaling study's v3 [5] examines architecture–task alignment. We cite that version without importing numerical claims from earlier versions. Hi-Agreement [9] investigates honest and deceptive testimony in distributed fact recovery, motivating separate false-assertion controls here. D3MAS [10] is relevant to hierarchical knowledge organization, but its arXiv page carries an author-withdrawal notice concerning methodology; we do not rely on its reported performance.

Behavioral invariance and directional testing are established by CheckList [8]. PAWS [11] illustrates the distinction between surface overlap and semantic equivalence. Metadata-sensitive conflict resolution [3] motivates explicit version-authority rules, while position effects in long contexts [7] motivate length and ordering controls. These alternatives must be considered before attributing failures to duplication itself.

3 Benchmark and Executable Truth

3.1 Candidate Schedule Selection

Each primary instance contains jobs with positive integer durations, nonnegative resource demands, resource capacities, an acyclic precedence graph, and candidate start times. Optional releases, deadlines and downtime are supported. Jobs execute nonpreemptively on half-open intervals $[s_j, s_j + d_j)$. For each resource r,

$$\sum_{j : s_j \le t < s_j + d_j} q_{jr} \le C_r \tag{1}$$

must hold at every event interval. Precedence requires $s_u + d_u \le s_v$ for every edge (u, v). Releases and deadlines bound starts and completions. A positive resource demand may not overlap that resource's downtime.

Let $F(c)$ be the conjunction of all mandatory predicates for candidate c, and $M(c) = \max_j(s_j + d_j)$. The answer minimizes $M(c)$ over supplied candidates with $F(c) = 1$. A public priority ordering resolves equal objectives; NONE is correct when the feasible set is empty. This is candidate selection, not unrestricted optimal scheduling. The event-sweep verifier is cross-checked against a separately coded integer-time occupancy checker. Tests also cover exact interval boundaries, deadline equality, invalid inputs, ties and NONE.

3.2 Paired Changes and Secondary Workflow

Original and modified worlds share candidate identifiers and controlled structure. Every pair records changed fields, mutation preconditions, affected predicates, oracle intermediate results, and both correct answers. Decision-changing and decision-preserving pairs are labeled by executable outcomes rather than intent. Scheduling currently implements capacity, resource identity and precedence edits. A separate finite Boolean workflow implements quantifiers, conditions and explicit version selection. The latter is reported as a secondary task family, not as scheduling evidence.

Generation randomizes identifiers, report order, candidate ordering and tie priority without consulting a model. Development and validation differ in parameter ranges; evaluation additionally reserves fork–join structure and rendering forms. All variants of one base problem remain in the same split. The construction is deliberately small and controlled; its limited graph and constraint diversity restricts external validity.

3.3 Scope, Truth and Ancestry

A typed claim carries task, candidate, entity, predicate, value, units, applicability, version, source identifiers, parent reports, verification execution and operation type. Scope equivalence excludes ancestry and asserted value, so compatible claims can still conflict. Copies preserve the source and execution; fresh verification can have a distinct execution while retaining the same source.

The task dependency graph, organizational hierarchy, communication/access graph, and report/evidence ancestry graph are stored separately. Scheduling job precedence is

an additional graph, not the graph of verification assignments. Evaluator-only mutation and correctness labels are kept outside model prompts. Shared-state action races and destructive concurrent tool effects are excluded.

4 Replay and Aggregation Protocol

4.1 Reports and Organization

Template reports are frozen from typed assertions before inference. The corpus contains no model-generated paraphrases. Code review and integrity tests check claim preservation; human linguistic review remains an open review item. The repetition conditions show a selected report once, four times exactly, or once with three templated paraphrases. No required finding is removed. A prespecified rotation chooses the repeated candidate/task, preferring an unchanged report across worlds. Version changes may require a declared exception.

The flat organization uses one root call with the complete report set. The hierarchy uses two managers with frozen partitions and one root; copies initially remain in the original branch. The root receives the summaries and a value-free registry of required claim scopes. The registry is available to every policy, including ordinary synthesis, and must be considered part of the evaluated interface. It does not reveal which assertions are true.

4.2 Policies and Observable Outputs

Ordinary synthesis states the same feasibility and optimization rule as the checklist and scope-aware policies. The checklist asks for required-task coverage. Scope-aware instructions additionally preserve entities, conditions, versions, unique findings and unresolved conflicts while consolidating equivalent ancestry. A challenge is not an automatic veto; a timestamp alone does not authorize a correction. The proposed policy has no privileged truth labels.

Final outputs contain a choice, class probabilities, explicit supporting assertions and unresolved scope keys. Intermediate outputs use the same assertion contract without a final choice. We score emitted claims, not private reasoning. The initial development interface required strict JSON but frequently produced Markdown fences and incorrect field types. Those outputs remain failures under that protocol. A second development interface uses named assertion fields and a common JSON schema decoder for all policies. This is an interface revision, not a repair that retrospectively changes the initial run's primary scores. Subsequent development revisions make scope identifiers descriptive, map objective labels explicitly to makespan or cost, and elicit a short feasibility/objective assessment for each candidate before the final choice. These assessments are model outputs, not oracle values. This common scaffold narrows the difference between ordinary synthesis and the checklist. A role-specific schema constrains managers' choice to null and their final-assessment fields to empty collections, enforcing their summary-only contract. The failed earlier manager outputs remain recorded. Predicate criteria are recorded separately from applicability: a failed resource check still

applies. A tokenizer audit also found an extra beginning-of-sequence token in historical Nemo calls. Those checks characterize their original interface, not clean model rankings. Subsequent inference submits native token IDs directly and freezes dates inserted by chat templates.

5 Execution Integrity and Analysis

Model revisions, tokenizer revisions, chat-template hashes, precision, dependency versions, sampling settings and output limits are recorded. Inference uses only the allocated NVIDIA RTX 6000 Ada GPU. No paid model API is used. Raw requests and responses are immutable; accepted calls and base-cluster completions are journaled. Cache keys include stochastic repetition identifiers. Resume reuses completed malformed outputs instead of sampling until valid. Interrupted, unaccepted calls are identifiable and are not silently treated as completed failures or successes. Local result mirrors prevent the pod from being the sole copy of completed data.

Let $E_{p,a,r}$ denote decision error averaged within base pair p, organization a and repetition condition r. Repetition harm is

$$H_{a,r} = \mathbb{E}_p[E_{p,a,r} - E_{p,a,\text{single}}]. \tag{2}$$

The hierarchy interaction is $H_{\text{hierarchical},r} - H_{\text{flat},r}$. A higher overall hierarchical error rate is not itself amplification of repetition. Correct responsiveness and correct stability require both paired answers to be correct, respectively on changing and preserving pairs. Merely switching or retaining a choice is insufficient.

The initial analysis manifest defines exact-copy and paraphrase effects (H1), the difference in pair-failure effects between consequential and harmless edits (H2), the hierarchy interaction (H3), and scope-versus-checklist error and retention contrasts (H4). Analysis resamples complete base clusters, stratified by mutation family and consequence, retaining dependence across worlds, arms, models and stochastic repetitions. Model-specific results are mandatory; a pooled estimate uses equal model weights. Scheduling and workflow estimates remain separate. No confirmatory significance claim is based on the development runs. Collapsed empirical bootstrap intervals are flagged: no observed variation in a small sample does not establish zero population effect or justify a power claim.

Malformed completed trajectories count as operational failures in primary accuracy. Calibration uses valid complete probability vectors and reports their availability fraction. Additional outcomes include invalid selections, assignment and delivery coverage, summary retention, scope transfer, copied-claim consolidation, false-assertion adoption and resource use. Input/output tokens and elapsed batch time are measured. Allocating batch time across requests is a cost-accounting convention, not a direct measurement of per-request GPU kernel time. Equal integer and integral-float objective values count as equivalent numeric claims; Boolean truth remains distinct from numeric one. Claim-score corrections are recomputed from immutable responses with a versioned scorer. An incompatible scope-citation metric records assertions supported by multiple reports whose scopes do not all bind that assertion; this is an emitted-attribution measure. False-assertion emission is separated from an uncontested false finding: the latter requires the

false value, absence of the contrary true value, and no unresolved flag for that scope. Neither establishes a hidden belief or causal reasoning path.

6 Development Results and Current Limits

Table 1. Completed development checks: correct/decisions (percent), ordinary, flat, single presentation. Checks reuse development pairs. Historical Mistral-7B/Nemo rows reflect their original tokenizer interface and are not clean model rankings.

Model	Interface	Scheduling	Workflow
Mistral-7B	v4	20/48 (41.7)	23/48 (47.9)
Mistral-7B	v5	31/48 (64.6)	33/48 (68.8)
Mistral-Small-24B (INT8)	v7	33/48 (68.8)	32/48 (66.7)
Mistral-Nemo-12B	v6	40/48 (83.3)	45/48 (93.8)
Mistral-Nemo-12B	v5	40/48 (83.3)	45/48 (93.8)
Mistral-Nemo-12B	v7	39/48 (81.2)	41/48 (85.4)
Phi-4 (BF16)	v7	48/48 (100.0)	45/48 (93.8)
Qwen2.5-14B	v2	16/48 (33.3)	37/48 (77.1)
Qwen2.5-14B	v3	32/48 (66.7)	32/48 (66.7)
Qwen3-14B	v3	26/48 (54.2)	30/48 (62.5)
Qwen3-14B	v4	32/48 (66.7)	29/48 (60.4)
Qwen3-14B	v5	48/48 (100.0)	42/48 (87.5)
Qwen3-14B	v7	47/48 (97.9)	42/48 (87.5)
Qwen2.5-7B	v2	15/48 (31.2)	25/48 (52.1)

Table 1 is regenerated from immutable-run analyses. These checks use ordinary, flat, single-presentation inputs and therefore do not estimate a duplication effect. The prespecified competence gate requires sufficient basic task performance before treating repetition failures as informative. Operational formatting and decision errors are recorded separately. Incomplete runs are not included in the table.

The initial replay was stopped during development after discovering a systematic interface failure. Its complete clusters and partial next cluster are retained, but it supplies no valid evidence for the scientific hypotheses. The revised competence checks determine the next experimental allocation. Main held-out comparisons, targeted model controls and live-worker results must not be inferred from the implemented code alone.

In the completed Qwen3 pilot, flat scheduling accuracy reached the ceiling across both policies and all repetition conditions. Hierarchical scheduling accuracy improved with copies and paraphrases under both policies. The negative hierarchy interaction therefore describes a larger repetition benefit in the hierarchy; it is not harmful amplification. Workflow estimates were mixed, and the paired responsiveness interaction remained imprecise. These development findings neither confirm a general harmful-duplication mechanism nor establish robustness. Scope-aware consolidation was not included in this pilot, so it cannot answer H4. The frozen development prefix contains ties but no NONE worlds; validation introduces that stratum. Its competence gate therefore does not establish performance on no-feasible-candidate decisions.

Table 2. Completed development pilot accuracy (percent); not a held-out hypothesis test. Each cell retains both worlds and both stochastic repetitions.

Task	Organization	Policy	Single	Exact	Paraphrase
schedule	flat	ordinary	100.0	100.0	100.0
schedule	flat	checklist	100.0	100.0	100.0
schedule	hierarchical	ordinary	81.2	95.8	95.8
schedule	hierarchical	checklist	81.2	93.8	87.5
workflow	flat	ordinary	87.5	83.3	87.5
workflow	flat	checklist	87.5	87.5	91.7
workflow	hierarchical	ordinary	83.3	85.4	87.5
workflow	hierarchical	checklist	79.2	81.2	83.3

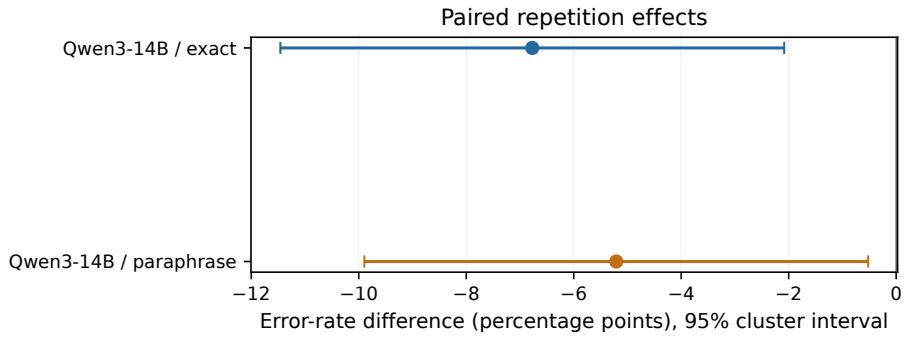


Fig. 1. Development-only primary scheduling: repetition. Intervals, where shown, resample base-instance clusters.

6.1 Dated Pilot Mechanism Audit

After the pilot and the original held-out freeze, the revised question asks when repetition protects information from summary loss and when it obscures meaningful differences. The dated amendment preserves H1–H4 and their original analyses; new mechanism analyses of those runs are exploratory. No original held-out model choices were inspected to design the amendment. Its new confirmatory contrasts require disjoint, prospectively frozen instances.

The pilot comprises 24 independent base pairs, not 1,152 independent observations. All repeated reports are correct. No selected objective favors a fastest infeasible candidate. Mandatory Boolean retention is near ceiling; numerical objectives sometimes become Boolean values or disappear despite syntactically valid JSON. Other errors occur after the correct failed constraint reaches the root. Therefore the pilot does not identify a single retention mechanism.

Unexposed-manager prompts and requested seeds matched in 192 paired comparisons per population. Normalized findings nevertheless differed in two workflow comparisons and none in scheduling. New tests share identical manager computations within a world and stochastic repetition, including the bypass control. This is a new coupling intervention, not a retrospective repair of pilot outputs.

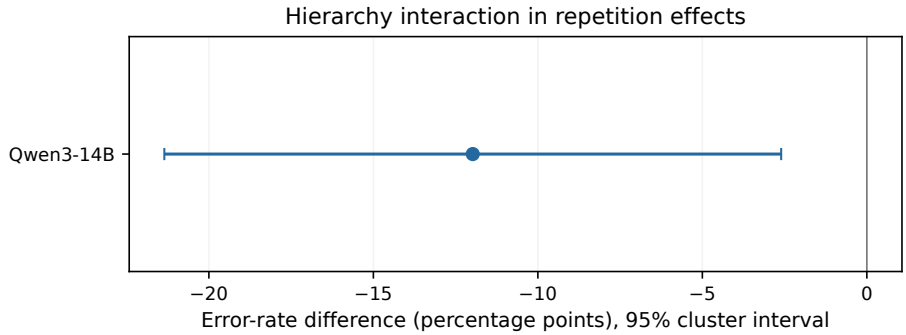


Fig. 2. Development-only primary scheduling: hierarchy. Intervals, where shown, resample base-instance clusters.

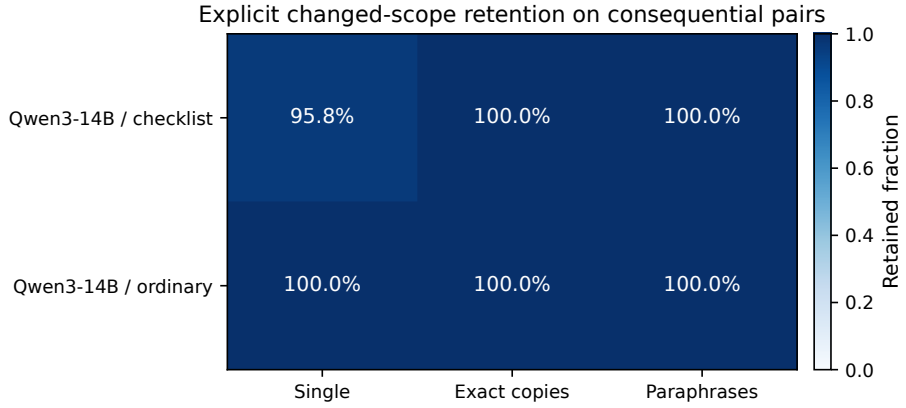


Fig. 3. Development-only primary scheduling: retention. Intervals, where shown, resample base-instance clusters.

The provisional design includes separate pilot, validation and held-out splits, two stochastic repetitions, and two model families. Actual manifests verify call counts. Final held-out allocation must be frozen after development and validation precision/runtime assessment, before inspecting held-out outcomes. No favorable significance threshold justifies stopping. A model that fails the competence gate is a documented limitation, not a candidate selected for a desirable failure rate.

7 Held-Out Replay Status

The held-out replay does not yet have a complete analysis. No numerical held-out claims are supplied.

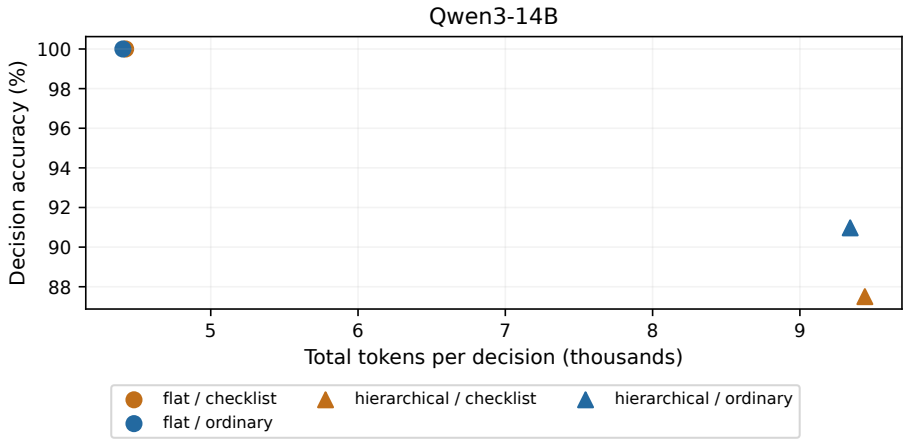


Fig. 4. Development-only primary scheduling: resource. Intervals, where shown, resample base-instance clusters.

8 Supporting Evaluation Status

The supporting held-out evaluation does not yet have a complete analysis.

9 Live Evaluation Status

The live held-out evaluation does not yet have a complete analysis.

10 Prospective Mechanism Extension

The amendment separates decisive, nondecisive, limited favorable and resolvable false findings, uniform repetition, and no added repetition. Development uses one model and 24 reused base pairs; all wider searches are exploratory. The narrow fresh-verification probe shares an initial report and source data across conditions and makes no independence claim. Fresh held-out allocation uses separate validation, a fixed precision/resource rule and disjoint identifiers.

The three prospective scheduling contrasts are decisive versus common-finding repetition (correctness), common-finding repetition versus single presentation (unrepeated necessary-claim retention), and the repetition-by-raw-access interaction (correctness). Bypass input growth and observed matching residuals limit mechanism attribution.

mechanism development v1: no complete model analysis is available; no numerical result is claimed.

fresh probe v1: no complete model analysis is available; no numerical result is claimed.

mechanism validation v1: no complete model analysis is available; no numerical result is claimed.

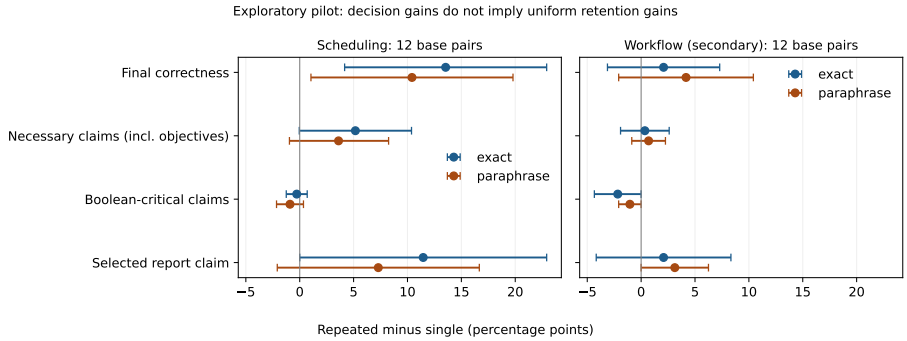


Fig. 5. Exploratory pilot: paired changes in correctness and explicit retention. Intervals resample independent base pairs, stratified by mutation family and change status. Necessary scopes include numerical objectives; Boolean-critical scopes use a truth-value-flip definition. These metrics do not observe hidden reasoning.

mechanism heldout v1: no complete model analysis is available; no numerical result is claimed.

11 Supporting Design and Threats to Validity

The supporting design contrasts overlapping and less-overlapping presentations with identical distinct claims, exact removal versus similarity removal, scope-aware consolidation, branch placement, raw-report bypass, and global reviewers matched to the hierarchy's call count. Neutral material controls for length; ancestry withholding and corrupted relationships test provenance use. Instruction-only controls separate metadata from additional prompting. Distinct information, ordering and report slots are held fixed where possible, with residual token differences recorded. The result sections explicitly identify execution status. Placement of distinct scoped reports and placement of exact copies are separate contrasts. Moving a report changes branch load, which is recorded as a residual difference. Similarity removal currently uses character overlap, so its findings cannot be generalized to learned semantic deduplication.

Corrupted reports require visible authoritative evidence or an allowed insufficient-information answer. False favorable assertions and false objections must be separated. Live workers receive role-specific task data without peer reports; four essential assignments remain in principal comparisons. Redundant fresh execution differs from frozen copying. Fixed output allowances alone do not match total computation, and coverage displacement requires its own label.

Major validity threats include small synthetic task diversity, structured scope registries that assist all policies, template-based language, restricted model families, formatting constraints, limited stochastic repetitions, and output caps. Shared weight families and shared source inputs can induce dependent errors. Retention associations alone cannot establish faithful internal reasoning or causal mediation. Summary bypass offers a direct intervention, but its additional input and compute must also be reported. Null

effects with wide intervals are inconclusive; beneficial hierarchy or sufficient simple checklists must be reported with the same prominence as harmful effects.

12 Conclusion

The implemented benchmark makes scope, ancestry, coverage and candidate feasibility explicit and auditable. Current development measurements constrain interpretation: interface validity and ordinary task competence must be established before the proposed duplication comparisons can support a paper. The contribution and final title remain conditional on the completed experiments and human scientific review.

Declaration on Generative AI and Review Status

The human author supplied the research direction and experimental requirements. An AI coding assistant helped implement and test the benchmark, operate local open-weight model experiments, retrieve literature, generate analyses, and draft text. Its methodological decisions and interpretations require author verification. This working draft has not yet undergone the author's scientific review. This disclosure must be updated truthfully after review; no AI is an author. The conference's limits on AI involvement require human assessment before submission.

References

1. Bara, M.: Epistemic sybil resistance: Multiplying AI agents without multiplying evidence. arXiv:2609.01873v1 (2026). URL https://arxiv.org/abs/2609.01873v1. Preprint
2. Cemri, M., Pan, M.Z., Yang, S., Agrawal, L.A., Chopra, B., Tiwari, R., Keutzer, K., Parameswaran, A., Klein, D., Ramchandran, K., Zaharia, M., Gonzalez, J.E., Stoica, I.: Why do multi-agent LLM systems fail? In: Advances in Neural Information Processing Systems, vol. 38 (2025). URL https://papers.nips.cc/paper_files/paper/2025/hash/b1041e52d3be19f0a9bc491657488e4a-Abstract-Datasets_and_Benchmarks_Track.html. Datasets and Benchmarks Track
3. Chiang, C.H., Lee, H.y.: Do metadata and appearance of the retrieved webpages affect LLM's reasoning in retrieval-augmented generation? In: Proceedings of the 7th BlackboxNLP Workshop, pp. 389–406 (2024). DOI 10.18653/v1/2024.blackboxnlp-1.24
4. Gao, J., Li, R., Yu, T., Ren, L., Chen, B., Wang, Z.: Copies or sources? measuring how LLM aggregators count restated evidence in multi-agent systems. arXiv:2610.06192v1 (2026). URL https://arxiv.org/abs/2610.06192v1. Preprint
5. Kim, Y., Gu, K., Park, C., Park, C., Schmidgall, S., Heydari, A.A., Yan, Y., Zhang, Z., Zhuang, Y., Liu, Y., Malhotra, M., Liang, P.P., Park, H.W., Yang, Y., Xu, X., Du, Y., Patel, S., Althoff, T., McDuff, D., Liu, X.: Towards a science of scaling agent systems. arXiv:2512.08296v3 (2026). URL https://arxiv.org/abs/2512.08296v3. Preprint, version dated 8 April 2026
6. Li, Y., Naito, A., Shirado, H.: Systematic failures in collective reasoning under distributed information in multi-agent LLMs. In: Proceedings of the 43rd International Conference on Machine Learning, *Proceedings of Machine Learning Research*, vol. 306, pp. 69,250–69,268. PMLR (2026). URL https://proceedings.mlr.press/v306/li26ej.html
7. Liu, N.F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., Liang, P.: Lost in the middle: How language models use long contexts. Transactions of the Association for Computational Linguistics **12**, 157–173 (2024). DOI 10.1162/tacl_a_00638
8. Ribeiro, M.T., Wu, T., Guestrin, C., Singh, S.: Beyond accuracy: Behavioral testing of NLP models with CheckList. In: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, pp. 4902–4912 (2020). DOI 10.18653/v1/2020.acl-main.442
9. Yan, C., Yue, Z., Zhao, F., Lin, E., Jia, L., Tong, H., Lyu, M., Sun, C., Zeng, Y.: When truth is distributed: Misinformation derails collective fact recovery in LLM-based multi-agent systems. arXiv:2608.03421v1 (2026). URL https://arxiv.org/abs/2608.03421v1. Preprint
10. Zhang, H., Shi, Y., Gu, X., You, H., Zhang, Z., Gan, L., Yuan, Y., Huang, J.: D3MAS: Decompose, deduce, and distribute for enhanced knowledge sharing in multi-agent systems. arXiv:2510.10585v3 (2026). URL https://arxiv.org/abs/2510.10585. Author withdrawal notice cites a fundamental methodological error; results not relied upon
11. Zhang, Y., Baldridge, J., He, L.: PAWS: Paraphrase adversaries from word scrambling. In: Proceedings of NAACL-HLT, pp. 1298–1308 (2019). DOI 10.18653/v1/N19-1131